TANSZÉKVEZETŐ

# SZAKDOLGOZAT FELADAT

**Szenes Márton Miklós**
Mérnökinformatikus hallgató részére

# Digitális becsületkassza alkalmazás fejlesztése közösségeknek

A közösségek és hallgatói körök működése során fontos feladat a pénzügyi folyamatok, a tagok egyenlegeinek és a körök által biztosított termékek nyilvántartása. Ezek kezelése manuálisan nehezen átlátható és időigényes lehet, ezért egy egységes digitális rendszer segítségével egyszerűsíthető a körök pénzügyeinek és a tagok fogyasztásának kezelése.

A hallgató feladata egy olyan webalkalmazás, a KasszaSCH elkészítése, amely lehetővé teszi a körök belső költségvetésének, a tagok digitális egyenlegének, valamint a körök által biztosított termékek és azok készletének kezelését. A rendszerben a felhasználók AuthSCH segítségével, illetve szükség esetén rendszergazda által létrehozott lokális fiókkal jelentkezhetnek be. Az alkalmazás különböző jogosultsági szinteket biztosít a felhasználók, a körvezetők és a rendszergazdák számára.

A körök tagjai megtekinthetik egyenlegüket és költéseiket, valamint az egyenlegük terhére termékeket vásárolhatnak. A körvezetők kezelhetik a kör termékeit, készletét és pénzügyeit, valamint a tagok egyenlegeit és a körön belüli jogosultságokat. A rendszer támogassa továbbá a körök közötti pénzmozgásokat, a tranzakciók kategorizálását és a tagok egyenlegének feltöltésére vonatkozó kérelmek kezelését.

A rendszergazdai felület biztosítson lehetőséget a körök és felhasználók kezelésére, a jogosultságok meghatározására, valamint a rendszer működésének nyomon követésére és auditálására. Az alkalmazás támogassa a körök létrehozásának és aktiválásának folyamatát, valamint a PÉK külső rendszerből származó kör- és tagsági adatok kezelését és szinkronizációját is.

A hallgató feladatának a következőkre kell kiterjednie:

- Mutassa be a KasszaSCH rendszer felé támasztott követelményeket!
- Válasszon alkalmas technológiákat a feladathoz és mutassa be azokat!
- Tervezze meg a rendszer architekturális felépítését!
- Indokoljon meg minden fontosabb tervezői döntést!
- Készítse el az alkalmazást!
- Gondoskodjon az alkalmazás konténerizálásáról!
- Igazolja az alkalmazás megfelelő működését automatikus funkcionális tesztekkel!

**Tanszéki konzulens:** Somogyi Norbert Zsolt, tanársegéd

Budapest, 2026. október 5.

Dr. Charaf Hassan
egyetemi tanár
tanszékvezető

Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Automatizálási és Alkalmazott Informatikai Tanszék
H-1117 Budapest, Magyar tudósok krt 2. Q.B.207
Telefon: 463-2870. Fax: 463-2871
www.aut.bme.hu




**Budapesti Műszaki és Gazdaságtudományi Egyetem**
Villamosmérnöki és Informatikai Kar
Automatizálási és Alkalmazott Informatikai Tanszék

# Digitális becsületkassza alkalmazás fejlesztése közösségeknek

Szakdolgozat

*Készítette*
Szenes Márton Miklós

*Konzulens*
Somogyi Norbert Zsolt

2026. október 13.

# Tartalomjegyzék

**Kivonat** **1**

**Abstract** **2**

**1. Bevezetés** **3**

**2. Felhasznált technológiák** **4**
2.1. Szerver (Backend) . . . 4
2.1.1. Python . . . 4
2.1.1.1. FastAPI . . . 4
2.1.1.2. PostgreSQL . . . 5
2.1.1.3. SQLAlchemy . . . 5
2.1.1.4. Alembic . . . 5
2.1.1.5. Pydantic . . . 5
2.1.2. Object Storage . . . 5
2.1.3. Representational State Transfer (REST) . . . 5
2.1.4. JSON Web Token (JWT) . . . 5
2.1.5. OAuth2.0 . . . 5
2.1.6. Docker . . . 5
2.2. Kliens (Frontend) . . . 5
2.2.1. React Native . . . 5
2.2.2. Expo . . . 5

**3. Tervezés** **6**
3.1. Specifikáció (Követelményelemzés) . . . 6
3.2. Architektúrális döntések . . . 6
3.2.1. Microservice vs Modular Monolith . . . 6
3.2.2. AuthSCH tagságok szinkronizálása . . . 6
3.3. UI tervek . . . 6

**4. Megvalósítás (Implementáció)** **7**
4.1. Szerver (backend) . . . 7
4.1.1. Adatbázis modell . . . 8
4.1.2. Modular monolith architektúra . . . 8
4.1.3. Modulok részletes felépítése . . . 10
4.1.3.1. Authentikáció . . . 10
4.1.3.2. Authorizáció . . . 10
4.1.3.3. Felhasználók . . . 10
4.1.3.4. Csoportok . . . 10
4.1.3.5. Csoporttagságok . . . 10
4.1.3.6. Szerepkörök . . . 10

4.1.3.7. Termékek . . . 10
4.1.3.8. Ikonok . . . 10
4.1.3.9. Bejegyzések . . . 10
4.1.3.10. Címkék . . . 10
4.1.3.11. Tranzakciók . . . 10
4.1.3.12. Beállítások . . . 10
4.1.3.13. Értesítések . . . 10
4.1.3.14. Statisztikák . . . 10
4.1.3.15. Audit logolás . . . 10
4.2. Kliens (frontend) alkalmazás . . . 10
4.2.1. Felhasználói oldalak . . . 10
4.2.1.1. Vásárlás . . . 10
4.2.1.2. Költések . . . 10
4.2.1.3. Utalás . . . 10
4.2.1.4. Áttekintés . . . 10
4.2.1.5. Pult . . . 10
4.2.1.6. Tagok . . . 10
4.2.1.7. Termékek . . . 10
4.2.1.8. Ikonok . . . 10
4.2.1.9. Címkék . . . 10
4.2.1.10. Tranzakciók . . . 10
4.2.1.11. Szerepkörök . . . 10
4.2.1.12. Meghívók . . . 10
4.2.2. Admin oldalak . . . 10
4.2.2.1. Csoportkérelmek . . . 10
4.2.2.2. Csoportok . . . 10
4.2.2.3. Globális szerepkörök . . . 10
4.2.2.4. Felhasználók . . . 10
4.2.2.5. Rendszerbeállítások . . . 10
4.2.2.6. Statisztikák . . . 10
4.2.2.7. Auditnapló . . . 10
4.2.3. Mobilapplikáció . . . 10

**5. Deployment** **11**
5.1. Konténerizáció . . . 11
5.2. CI/CD . . . 11
5.3. Staging és Production környezet . . . 11

**6. Tesztelés** **12**

**7. Összefoglalás** **13**
7.1. Továbbfejlesztési lehetőségek . . . 13
7.2. Tapasztalatok . . . 13

**Irodalomjegyzék** **14**

**Függelék** **15**
F.1. Nyilatkozat generatív mesterséges intelligencia alkalmazásáról . . . 15

## HALLGATÓI NYILATKOZAT

Alulírott *Szenes Márton Miklós*, szigorló hallgató kijelentem, hogy ezt a szakdolgozatot meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem. A Függelékben egyértelműen megjelöltem, hogy a mesterséges intelligencia eszközeit alkalmaztam-e a dolgozat elkészítéséhez; amennyiben igen, annak módját és mértékét a táblázatban közöltem. Tudomásul veszem, hogy a mesterséges intelligenciával generált tartalomért – annak mértékétől függetlenül – teljes felelősséggel tartozom.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövege pedig a BME Címtárban regisztrált személyek számára elérhető legyen. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Budapest, 2026. október 13.

*Szenes Márton Miklós*
hallgató

# Kivonat

A különféle közösségek és diákkörök mindennapi működésük, belső rendezvényeik és táboraik során számos költségmegosztási és készletkezelési feladattal szembesülnek: a közösségi készletek felhasználását, a tagok közös kiadásait, az előzetes költség-hozzájárulásokat, valamint a csoportos kiadásokat nyomon kell követni. Ezt a belső elszámolást a csoportok jellemzően papíron, táblázatkezelőben vagy egyszerű feljegyzésekkel vezetik. Ezek a módszerek nehezen átláthatók, hibalehetőségeket hordoznak, nem biztosítanak valós idejű egyenlegkövetést a tagoknak, és utólag nehezen rekonstruálható a tranzakciók pontos története.

A KasszaSCH célja, hogy erre a problémára egységes, megbízható és könnyen használható digitális elszámolási és nyilvántartási megoldást nyújtson. Ennek megoldásaként a rendszer nyilvántartja a csoportok belső költségvetését, az elérhető termékek készletét, azok ikonjait, támogatja a tagoknak a termékek megvásárlását, tranzakciók hozzárendelését tagokhoz, a tagok számára közösségenként elkülönített belső egyenleget biztosít a költések követésére, és egy adminisztrációs felületet mindezek monitorozására.

# Abstract

Various communities and student groups face a lot of expense-sharing and inventory tasks during their everyday activities, internal events, and camps. They need to track communal supplies, shared expenses, upfront contributions, and group spending. Groups usually handle these accounts on paper, in spreadsheets, or with chat notes. However, these methods are hard to see through, prone to errors, don't let members check their balances in real time, and make it difficult to retrace the exact transaction history later on.

KasszaSCH aims to solve this problem by providing a unified, reliable, and easy-to-use digital accounting and inventory management system. As a solution, the system keeps track of group budgets, available item inventories, and their icons. It also supports member purchases, assigns transactions to specific members, offers separate balances for each community to follow spending, and provides an admin interface to monitor everything.

# 1. fejezet

# Bevezetés

Ahogy minden projekt, ez is egy problémával indult. Minden csoport vagy közösség eseményein előfordul a szükség arra, hogy a közösen megvásárolt termékeket a közösségen belül elfogyaszthassák a tagok, és becsületkassza alapon bedobják az összegét az elvett terméknek. Ezt legtöbbször egy papíron vezetik strigulázva a nevek mellett vagy jobb esetben egy táblázatkezelő szoftverbe vezetik, hogy ki mit fogyasztott a közösből. Bár ezek megoldják az alapvető problémát, de nem tökéletes. Nem lehet velük visszamenőleg követni, hogy ki, mikor, mit csinált, mennyi az egyenlege, stb…

Ezért aztán jött az ötlet, hogy mérnökinformatikusként nehogy már papíron vezessük ezeket a kiadásokat, hanem legyen erre egy szoftver, ami ezt követni, és mindenféle okos funkcióval megkönnyíti és megoldja az említett problémát. Így született meg egy évvel ezelőtt a KasszaSCH ötlete.

Természetesen, nem ugrottam fejest rögtön a kódolásba, hiszen egy szoftver megtervezése előtt elő kell állni a követelményekkel, hogy mire lenne szükségük a felhasználóknak, kutatást végezni, hogy van-e erre már egy jól bevált program, vagy annyira speciálisak az igények, hogy egyszerűbb egy sajátot készíteni. Sok emberrel való konzultáció után megszülettek az igények, hogy mit is kéne tudjon a jövőbeli alkalmazás. Ezen belül megállapítottuk, mik lennének a legfontosabb funkciók, és mik a ”jó lenne, ha ezt is tudná” kérések.

Fél évvel később, a stakeholderekkel való hosszas beszélgetés, a követelmények elemzése és megfogalmazása, valamint időhiány után elkezdődött a KasszaSCH fejlesztése.

Dolgozatomban ennek a rendszernek a tervezését és megvalósítását fogom bemutatni.

# 2. fejezet

# Felhasznált technológiák

A projekt megvalósításához számos modern technológiát alkalmaztam, amelyek lehetővé tették a hatékony fejlesztést és egy skálázható rendszer kialakítását. A következőkben részletesen bemutatom az egyes technológiákat és azok szerepét a rendszerben.

## 2.1. Szerver (Backend)

### 2.1.1. Python

A Python [1] egy magas szintű, általános célú programozási nyelv, amelyet Guido van Rossum hozott létre 1991-ben. Könnyen olvasható szintaxisa és széles körű könyvtárkészlete miatt népszerű a webfejlesztés, adatfeldolgozás, gépi tanulás és sok más területen. A Python dinamikus típusosságot és objektumorientált programozást támogat, valamint lehetőséget biztosít aszinkron programozásra is. A projekthez a Python 3.13.3-as verzióját használtam.

#### 2.1.1.1. FastAPI

A backend (szerver) fejlesztéséhez a FastAPI [2] keretrendszert választottam, ami egy modern, gyors webkeretrendszer Python nyelven, amely lehetővé teszi REST-ful[1] API-k egyszerű fejlesztését. Fő előnye, hogy standard Python típusannotációkat használ és ezzel típusbiztonságot nyújt. Emellett aszinkron működést biztosít az endpointoknak az I/O-intenzív feladatoknál (pl. adatbázis lekérdesék, külső API hívások), gyors fejlesztést tesz lehetővé a beépített validáció és dokumentáció révén, és automatikus Swagger [3] dokumentációt is az endpointoknak[2].

[1] REST: Representational State Transfer

[2] Endpoint: (magyarul végpont) Egy digitális cím (általában egy URL), amin keresztül a szerver-alkalmazás (API - Application Programming Interface) fogadja a külső kéréseket, és válaszként adatot szolgáltat.

#### 2.1.1.2. PostgreSQL

#### 2.1.1.3. SQLAlchemy

#### 2.1.1.4. Alembic

#### 2.1.1.5. Pydantic

### 2.1.2. Object Storage

### 2.1.3. Representational State Transfer (REST)

### 2.1.4. JSON Web Token (JWT)

### 2.1.5. OAuth2.0

### 2.1.6. Docker

## 2.2. Kliens (Frontend)

### 2.2.1. React Native

### 2.2.2. Expo

# 3. fejezet

# Tervezés

## 3.1. Specifikáció (Követelményelemzés)

## 3.2. Architektúrális döntések

### 3.2.1. Microservice vs Modular Monolith

### 3.2.2. AuthSCH tagságok szinkronizálása

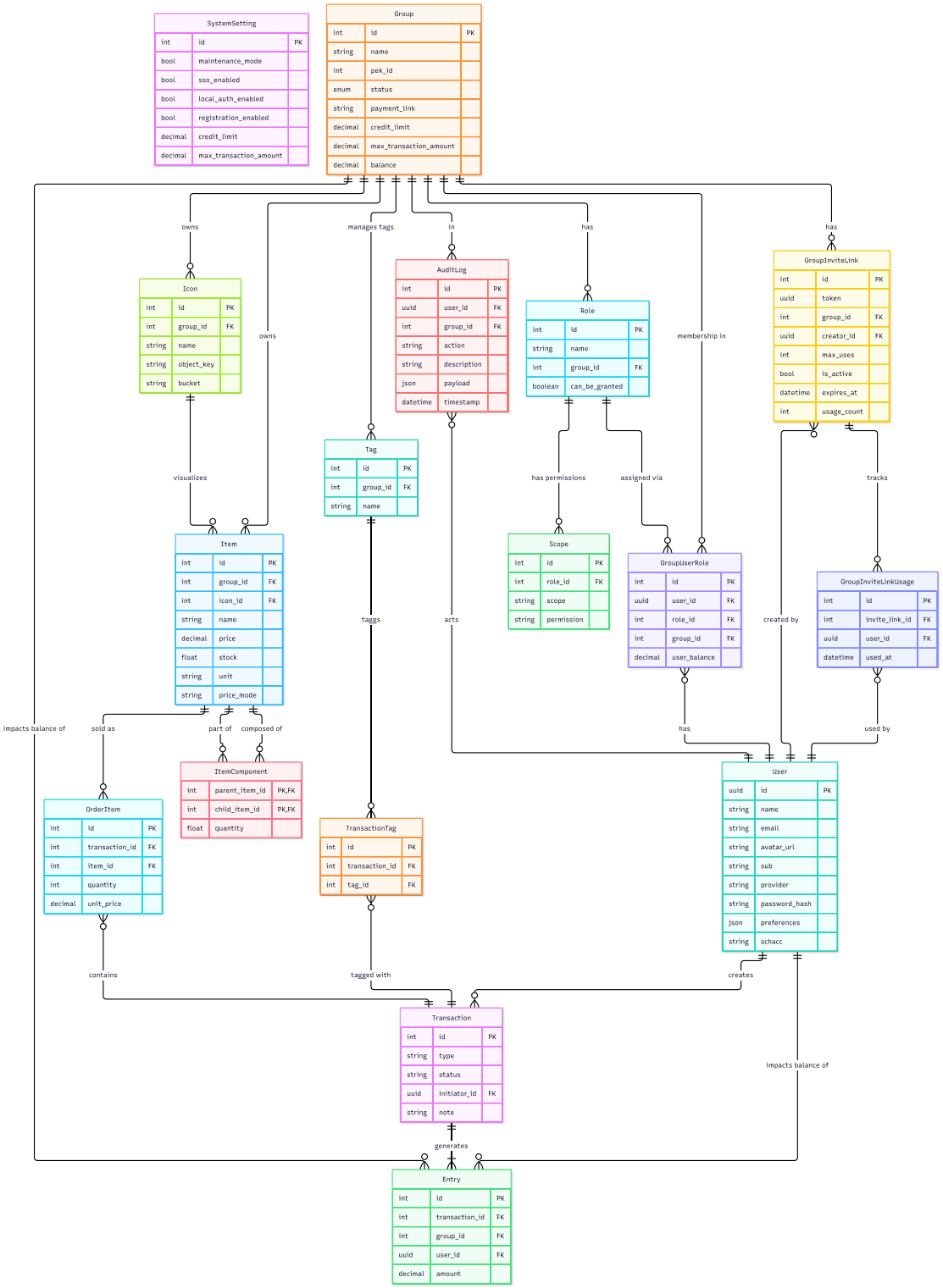
## 3.3. UI tervek

# 4. fejezet

# Megvalósítás (Implementáció)

## 4.1. Szerver (backend)

### 4.1.1. Adatbázis modell




**4.1. ábra.** Teljes adatbázis séma

### 4.1.2. Modular monolith architektúra

Azért lett modular monolith, mert...

#### 4.1.3. Modulok részletes felépítése

##### 4.1.3.1. Authentikáció

##### 4.1.3.2. Authorizáció

##### 4.1.3.3. Felhasználók

##### 4.1.3.4. Csoportok

##### 4.1.3.5. Csoporttagságok

##### 4.1.3.6. Szerepkörök

##### 4.1.3.7. Termékek

##### 4.1.3.8. Ikonok

##### 4.1.3.9. Bejegyzések

##### 4.1.3.10. Címkék

##### 4.1.3.11. Tranzakciók

##### 4.1.3.12. Beállítások

##### 4.1.3.13. Értesítések

##### 4.1.3.14. Statisztikák

##### 4.1.3.15. Audit logolás

### 4.2. Kliens (frontend) alkalmazás

#### 4.2.1. Felhasználói oldalak

##### 4.2.1.1. Vásárlás

##### 4.2.1.2. Költések

##### 4.2.1.3. Utalás

##### 4.2.1.4. Áttekintés

##### 4.2.1.5. Pult

##### 4.2.1.6. Tagok

##### 4.2.1.7. Termékek

##### 4.2.1.8. Ikonok

##### 4.2.1.9. Címkék

##### 4.2.1.10. Tranzakciók

##### 4.2.1.11. Szerepkörök

##### 4.2.1.12. Meghívók

#### 4.2.2. Admin oldalak

##### 4.2.2.1. Csoportkérelmek

##### 4.2.2.2. Csoportok

##### 4.2.2.3. Globális szerepkörök

##### 4.2.2.4. Felhasználók

##### 4.2.2.5. Rendszerbeállítások

##### 4.2.2.6. Statisztikák

# 5. fejezet

# Deployment

## 5.1. Konténerizáció

## 5.2. CI/CD

## 5.3. Staging és Production környezet

# 6. fejezet

# Tesztelés

# 7. fejezet

# Összefoglalás

## 7.1. Továbbfejlesztési lehetőségek

## 7.2. Tapasztalatok

# Irodalomjegyzék

[1] Python Software Foundation: Python - wikipedia (2026. október 13.). https://en.wikipedia.org/wiki/Python_(programming_language).

[2] Sebastián Ramírez: FastAPI - hivatalos okumentáció (2026. október 13.). https://fastapi.tiangolo.com.

[3] Swagger: Swagger - hivatalos dokumentáció (2026. október 13.). https://swagger.io.

# Függelék

## F.1. Nyilatkozat generatív mesterséges intelligencia alkalmazásáról

☐ **Nem használtam** semmilyen generatív MI segédeszközt.
☒ **Használtam** generatív MI segédeszközt. Az MI-vel generált tartalmakat ellenőriztem, a generált kimenetek valóságtartalmáról meggyőződtem, az alábbi táblázatban megfelelően jelöltem minden használatot.

| **Felhasználási módok** | **Generatív MI eszköz(ök) neve** | **Érintett részek** (fejezet, oldalszám, hivatkozás) | **Használat becsült aránya** (felhasználási módonként) |
|---|---|---|---|
| Irodalomkutatás | | | |
| **Prompt lényegi része** | | | |
| Programkód generálása | | | |
| **Prompt lényegi része** | | | |
| Új ötletek, megoldási javaslatok generálása | | | |
| **Prompt lényegi része** | | | |
| Vázlat létrehozása (szövegstruktúra, vázlatpontok) | | | |
| **Prompt lényegi része** | | | |
| Szövegblokkok létrehozása | Gemini | Abstract | 80% |
| **Prompt lényegi része** | Összefoglaló fejezet angolra fordítása. | | |
| Képek generálása illusztrációs célból | | | |
| **Prompt lényegi része** | | | |
| Adatvizualizáció, grafikonok generálása adatpontok alapján | | | |
| **Prompt lényegi része** | | | |
| Prezentáció készítése | | | |
| **Prompt lényegi része** | | | |
| Egyéb (nevezze meg) | | | |
| **Prompt lényegi része** | | | |
| **Összesített százalékos érték (a feladat érdemi részére nézve):** | | | |
| **Összesített érték rövid, szöveges indoklása:** | | | |